

多行输入为一个列表套列表

```
n=int(input())
l=[]
for i in range(n):
    l.append(list(map(int,input().split())))
print(l)
```

结果

```
5
10 4
13 9
100 13
123 456
92 46
[[10, 4], [13, 9], [100, 13], [123, 456], [92, 46]]
```

输入为一个大列表

```
n=int(input())
l=[]
for i in range(n):
    l.extend(map(int,input().split()))
print(l)
```

结果

```
5
1 2
2 4
5 1
3 8
12 98
[1, 2, 2, 4, 5, 1, 3, 8, 12, 98]
```

输入为大列表但是字符串

```
n=int(input())
l=[]
for i in range(n):
    l.extend(input().split())
print(l)
```

结果

```
5
1 2
2 4
5 1
3 8
12 98
['1', '2', '2', '4', '5', '1', '3', '8', '12', '98']
```

汉诺塔

```
def hanoi(n,a,b,c):
    if n>0:
        hanoi(n-1,a,c,b)
        print("moving from %s to %s" % (a,c))
        hanoi(n-1, b, a, c)

n=int(input())
hanoi(n, 'A','B','C')
```

二分查找

```
def binary_search(li, val):
    left = 0
    right = len(li) - 1
    while left <= right: # 候选区有值
        mid = (left + right) // 2
        if li[mid] == val:
            return mid
        elif li[mid] > val: # 待查找的在mid左侧
            right = mid - 1
        else:
            left = mid + 1
    else:
        return None

li=list(map(int,input().split()))
l=sorted(li)
val=int(input())
i=binary_search(l,val)
print(li.index(l[i]))
```

线性查找

```
def linear_search(li,val):
    for index, value in enumerate(li):
        if value==val:
            return index

    else:
        return None

li=[5, 6, 6, 3, 5, 1, 7, 9]
val=int(input())
print(linear_search(li,val))
```

冒泡排序

每一趟相邻的两数比较，按大小交换位置，小在前大在后

每跑一趟，有序区增加一个数，无序区减少一个数。

```
def bubble_sort(li):
    for i in range(len(li)-1): #跑多少趟
        for j in range(len(li)-i-1): #指针位置
            if li[j]>li[j+1]:
                li[j], li[j+1]=li[j+1], li[j]
```

优化后

```
def bubble_sort(li):
    for i in range(len(li)-1): #跑多少趟
        exchange = False
        for j in range(len(li)-i-1): #指针位置
            if li[j]>li[j+1]:
                li[j], li[j+1]=li[j+1], li[j]
                exchange = True
        print(li)
    if not exchange: #避免本来升序又排了一遍
        return
```

栈

```
class Stack:
    def __init__(self):
        self.stack=[]
    def push(self, element):
        self.stack.append(element)
    def pop(self):
        return self.stack.pop()
    def get_top(self):
        if len(self.stack)>0:
            return self.stack[-1]
        else:
            return None
```

堆

先写调整顺序的函数：最大的一直在最上面

```
def sift(li, low, high): #li列表 low根节点的位置 high堆最后一个元素的位置
    i = low #i最开始先指向堆顶（根节点）随着循环进行下移
    j = i*2+1 #j是i的左孩子
    tmp = li[low] #把堆顶存起来
    while j<=high: #只要j指向的位置有数
        if j+1<=high and li[j+1]>li[j]: #右孩子存在且更大
            j=j+1 #指向右孩子但不能交换，万一交换了不比下方的大
        if li[j]>tmp:
            li[i]=li[j]
            i=j #往下一层
```

```

        j=2*i+1
    else:        #tmp更大, 把tmp放在i的位置上
        li[i]=tmp
        break
else:
    li[i]=tmp

```

再写建堆的函数

```

def heap_sort(li):
    n=len(li)
    for i in range((n-2)//2, -1, -1):#对最后的那个和他的父亲, i表示建堆的时候“调整的部分”的根的下标
        sift(li, i, n-1)#投机取巧, 直接令high是整个堆的最后一个
        #建堆成功#这里建的是大根堆, 如果要建小根堆把sift函数中***行的不等号调换
    for i in range(n-1, -1, -1):#挨个出数
        li[0], li[1] = li[1], li[0]
        sift(li, 0, i-1)#i-1是新的high
    return li

```

运用内置模块heapq

```

li=[1,2,2,7,8,12,3,9,6]
import heapq
heapq.heapify(li)#生成小根堆
heapq.heappop(li)#弹出最小的那一项
heapq.heappush(li, item)#把item压入li中并保持性质不变
heapq.heappushpop(li, item)#先压入再弹出最小, 有可能弹出压入值
heapq.heapreplace(li, item)#先弹出最小再压入item
heapq.heapnlargest(li, n)#找出前n大的

```

动态规划

```

#钢条切割问题

def cut_dp(p,n):
    r=[0]
    for i in range (1,n+1):
        res=0
        for i in range(1,i+1):
            res=max(res, p[j]+r[i-j])
        r.append(res)

```

动态规划写斐波那契

```

#递归长时间
def fib(n):
    if n==1 or n==2:
        return 1
    else:
        return fib(n-1)+fib(n-2)
#dp实现:最优子结构(递推式)
def fib_dp(n):

```

```
f = [0,1,1]
if n>2:
    for i in range(n-2):
        num = f[-1]+f[-2]
        f.append(num)
return f[n]
```

贪心

```
#数字拼接问题
li=list(map(int,input().split()))
from functools import cmp_to_key
def xy_cmp(x, y):
    if x+y < y+x:
        return 1
    elif x+y > y+x:
        return -1
    else:
        return 0
def number_join(li):
    li = list(map(str, li))
    li.sort(key=cmp_to_key(xy_cmp))
    return "".join(li)
print(number_join(li))
```

一些语法

1.保留小数

```
num=1.30989095
print(round(num,5)) #不会补充0
num0=f"{float(num):.5f}" #可以补充零
```

2.进制转换

```
bin()#10转2，但开头会有0b
oct()#10转8，开头有0o
hex()#10转16，开头会有0x
```

3.ASCII表

```
ord()#字符转数字
chr()#数字转字符
```

4.math模块

```

import math
print(math.ceil(1.5)) # 2
print(math.pow(2,3)) # 8.0算幂但是生成浮点数
print(math.pow(2,2.5)) # 5.656854249492381
print(9999999>math.inf) # False
print(math.sqrt(4)) # 2.0
print(math.log(100,10)) # 2.0  math.log(x,base) 以base为底，x的对数
print(math.comb(5,3)) # 组合数，C53
print(math.factorial(5)) # 5!

```

5.二分查找

```

import bisect
sorted_list = [1,3,5,7,9] #[(0)1, (1)3, (2)5, (3)7, (4)9]
position = bisect.bisect_left(sorted_list, 6)
print(position) # 输出: 3, 因为6应该插入到位置3, 才能保持列表的升序顺序

bisect.insort_left(sorted_list, 6)
print(sorted_list) # 输出: [1, 3, 5, 6, 7, 9], 6被插入到适当的位置以保持升序顺序

sorted_list=(1,3,5,7,7,7,9)
print(bisect.bisect_left(sorted_list,7))
print(bisect.bisect_right(sorted_list,7))
# 输出: 3 6

```

6.双指针

```

#双指针找和相等的两个数
n = int(input())
a = list(map(int, input().split()))
M = int(input())
i = 0
j = n - 1
while i < j:
    if a[i] + a[j] == M:
        print(a[i], a[j])
        i += 1
        j -= 1
    elif a[i] + a[j] < M:
        i += 1
    else:
        j -= 1

```

```

#双指针解决融合成递增序列
def merge(A, B):
    i, j = 0, 0
    c = []
    # 合并两个有序数组
    while i < len(A) and j < len(B):
        if A[i] <= B[j]:
            c.append(A[i])
            i += 1
        else:

```

```

        c.append(B[j])
        j += 1
# 将 A 的剩余元素加入 c
        c.extend(A[i:])
# 将 B 的剩余元素加入 c
        c.extend(B[j:])

    return len(c), c
A = [1, 3, 5, 7]
B = [2, 4, 6, 8]
length, c = merge(A, B)
print(c)

```

6.素数筛&找到所有素因数

```

# 胡睿诚 23数院
N=20
primes = []
is_prime = [True]*N
is_prime[0] = False;is_prime[1] = False
for i in range(2,N):
    if is_prime[i]:
        primes.append(i)
        for p in primes: #筛掉每个数的素数倍
            if p*i >= N:
                break
            is_prime[p*i] = False
            if i % p == 0: #这样能保证每个数都被它的最小素因数筛掉!
                break
print(primes)
# [2, 3, 5, 7, 11, 13, 17, 19]

```

```

def pFactors(n):
    """Finds the prime factors of 'n'"""
    from math import sqrt
    pFact, limit, check, num = [], int(sqrt(n)) + 1, 2, n
    for check in range(2, limit):
        while num % check == 0:
            pFact.append(check)
            num /= check
    if num > 1:
        pFact.append(num)
    return pFact
#print(pFactors(12))
#[2, 2, 3]
#print(pFactors(1))
#[ ]
#print(pFactors(30013))
#[30013]

```

7.按某种方式分割

```
#输入6n^2+5n^3
#按照+分割
s = input().split('+')
```

8.字符串的操作

```
k="ahauicghb7yda998j"
k0=k.upper()#小写变大写，输出#AHAUICGHB7YDA998J#
k1=k.lower()#大写变小写
print(98 in k)#查找98是否在k中，输出True#列表同样适用
```

9.列表

```
l=[0]*5
print(l)#输出[0, 0, 0, 0, 0]
l=[[0]*5]
print(l)#[[0, 0, 0, 0, 0]]
l=[[0]*5]*5
print(l)#[[0, 0, 0, 0, 0], [0, 0, 0, 0, 0], [0, 0, 0, 0, 0], [0, 0, 0, 0, 0], [0, 0, 0, 0, 0]]
#返回索引值
l.index(n0)
#计算出现个数
l.count(n0)
#把列表输出成无空格字符串
print(''.join(map(str,l)))
#想找出一个列表里没有的另一个列表的元素
setquan=set(lquan)
setsheng=set(lsheng)
setcha=setquan-setsheng
lcha=list(setcha)
#有无重复元素，用set（）检查
```

10.lambda函数

```
add = lambda x, y: x + y
print(add(3, 5)) # 输出 8
```

```
# 使用 lambda 作为排序的键
points = [(1, 2), (4, 1), (5, 0)]
sorted_points = sorted(points, key=lambda p: p[1]) # 按 y 值排序
print(sorted_points) # 输出 [(5, 0), (4, 1), (1, 2)]
```

```
# 在 map 中使用 lambda
numbers = [1, 2, 3, 4]
squared = list(map(lambda x: x ** 2, numbers)) # 平方每个数字
print(squared) # 输出 [1, 4, 9, 16]
```



```
# 使用 filter 过滤列表
numbers = [1, 2, 3, 4, 5]
even_numbers = list(filter(lambda x: x % 2 == 0, numbers)) # 过滤出偶数
print(even_numbers) # 输出 [2, 4]
```

```
# 列表推导式中使用 lambda
numbers = [1, 2, 3, 4]
doubled = [lambda x: x * 2 for x in numbers] # 这是不正确的，输出是函数对象
print([f(n) for f, n in zip(doubled, numbers)]) # 正确用法
```

字典

1. 使用大括号 {}

```
my_dict = {"name": "Alice", "age": 25, "city": "New York"}
print(my_dict)
```

2. 使用 dict() 函数

```
my_dict = dict(name="Alice", age=25, city="New York")
print(my_dict)
```

3. 通过键访问值

```
my_dict = {"name": "Alice", "age": 25, "city": "New York"}
print(my_dict["name"]) # 输出: Alice
```

4. 使用 .get() 方法

- .get() 方法可以提供默认值，当键不存在时返回这个默认值，而不会抛出异常。

```
print(my_dict.get("name")) # 输出: Alice
print(my_dict.get("country", "Not Found")) # 输出: Not Found
```

5. 添加新键值对

```
my_dict["country"] = "USA"
print(my_dict) # 输出: {'name': 'Alice', 'age': 25, 'city': 'New York',
'country': 'USA'}
```

6. 修改已有键的值

```
my_dict["age"] = 26
print(my_dict) # 输出: {'name': 'Alice', 'age': 26, 'city': 'New York',
'country': 'USA'}
```

7. 使用 del 删除某个键值对

```
del my_dict["city"]
print(my_dict) # 输出: {'name': 'Alice', 'age': 26, 'country': 'USA'}
```

8. 使用 `.pop()` 删除并返回某个键的值

```
age = my_dict.pop("age")
print(age) # 输出: 26
print(my_dict) # 输出: {'name': 'Alice', 'country': 'USA'}
```

字典的遍历

1. 遍历字典的键

```
my_dict = {"name": "Alice", "age": 25, "city": "New York"}
for key in my_dict:
    print(key) # 输出: name, age, city
```

2. 遍历字典的值

```
for value in my_dict.values():
    print(value) # 输出: Alice, 25, New York
```

3. 遍历字典的键值对

```
for key, value in my_dict.items():
    print(f"{key}: {value}")
```

字典的其他常用方法

1. 获取所有键

```
keys = my_dict.keys()
print(keys) # 输出: dict_keys(['name', 'age', 'city'])
```

2. 获取所有值

```
values = my_dict.values()
print(values) # 输出: dict_values(['Alice', 25, 'New York'])
```

3. 获取所有键值对

```
items = my_dict.items()
print(items) # 输出: dict_items([('name', 'Alice'), ('age', 25), ('city', 'New York')])
```

4. 从字典中获取指定键的值, 如果键不存在, 则返回默认值

```
my_dict = {"name": "Alice", "age": 25}
print(my_dict.get("name", "Not Found")) # 输出: Alice
print(my_dict.get("city", "Not Found")) # 输出: Not Found
```

5. 使用 `setdefault()` 获取键的值, 若键不存在, 则插入一个默认值

```
print(my_dict.setdefault("city", "Unknown")) # 输出: Unknown
print(my_dict) # 输出: {'name': 'Alice', 'age': 25, 'city': 'Unknown'}
```

嵌套字典

字典可以嵌套在另一个字典中，可以用来表示更复杂的数据结构。例如，表示学生的信息：

```
students = {
    "Alice": {"age": 25, "city": "New York"},
    "Bob": {"age": 22, "city": "Los Angeles"}
}

print(students["Alice"]["age"]) # 输出: 25
```

字典推导式

字典推导式是 Python 中生成字典的简洁方式，类似于列表推导式。

```
# 创建一个字典，键是数字，值是其平方
squares = {x: x**2 for x in range(5)}
print(squares) # 输出: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

合并字典

1. 使用 `update()` 方法合并字典

```
dict1 = {"name": "Alice", "age": 25}
dict2 = {"city": "New York", "job": "Engineer"}
dict1.update(dict2)
print(dict1) # 输出: {'name': 'Alice', 'age': 25, 'city': 'New York', 'job': 'Engineer'}
```

2. 使用 `|` 运算符 (Python 3.9+) 合并字典

```
dict1 = {"name": "Alice", "age": 25}
dict2 = {"city": "New York", "job": "Engineer"}
dict3 = dict1 | dict2
print(dict3) # 输出: {'name': 'Alice', 'age': 25, 'city': 'New York', 'job': 'Engineer'}
```

enumerate 返回索引

`enumerate` 是 Python 的一个内置函数，常用于迭代可迭代对象（如列表、元组、字符串等）时，同时获取元素的索引和元素本身。

```
enumerate(iterable, start=0)
```

`enumerate` 返回一个迭代器，其中每个元素是一个包含两个值的元组，分别是索引和对应的元素。

基本用法

```
fruits = ['apple', 'banana', 'cherry']
for index, fruit in enumerate(fruits):
    print(index, fruit)
```

```
0 apple
1 banana
2 cherry
```

修改起始索引

```
fruits = ['apple', 'banana', 'cherry']
for index, fruit in enumerate(fruits, start=1):
    print(index, fruit)
```

```
1 apple
2 banana
3 cherry
```

与列表解析结合

可以使用 `enumerate` 和列表解析生成新的列表：

```
fruits = ['apple', 'banana', 'cherry']
indexed_fruits = [(i, fruit) for i, fruit in enumerate(fruits)]
print(indexed_fruits)
```

```
[(0, 'apple'), (1, 'banana'), (2, 'cherry')]
```

用于更新列表

```
numbers = [10, 20, 30]
for index, value in enumerate(numbers):
    numbers[index] = value * 2
print(numbers)
```

```
[20, 40, 60]
```

和字典结合使用

将列表转化为字典，其中索引作为键，元素作为值：

```
fruits = ['apple', 'banana', 'cherry']
fruit_dict = dict(enumerate(fruits))
print(fruit_dict)
```

```
{0: 'apple', 1: 'banana', 2: 'cherry'}
```

permutations全排列

```
itertools.permutations(iterable, r=None)
```

```
from itertools import permutations
```

如果 `r` 未指定，则生成输入所有元素的全排列：

可以通过设置 `r` 生成特定长度的排列：

```
from itertools import permutations

data = [1, 2, 3]
result = permutations(data, r=2)
print(list(result))
```

```
[(1, 2), (1, 3), (2, 1), (2, 3), (3, 1), (3, 2)]
```

用于字符串

`permutations` 也适用于字符串，生成的排列为字符的组合：

```
from itertools import permutations

data = 'ABC'
result = permutations(data)
print([''.join(p) for p in result])
```

与 for 循环结合

```
from itertools import permutations

data = [1, 2, 3]
for p in permutations(data, 2):
    print(p)
```

二进制	十进制	十六进制	缩写	解释	二进制	十进制	十六进制	字符	二进制	十进制	十六进制	字符	二进制	十进制	十六进制	字符
0000 0000	0	0	NUL	空字符(Null)	0010 0000	32	20	空格	0100 0001	65	41	A	0110 0001	97	61	a
0000 0001	1	1	SOH	标题开始	0010 0001	33	21	!	0100 0010	66	42	B	0110 0010	98	62	b
0000 0010	2	2	STX	正文开始	0010 0010	34	22	"	0100 0011	67	43	C	0110 0011	99	63	c
0000 0011	3	3	ETX	正文结束	0010 0011	35	23	#	0100 0100	68	44	D	0110 0100	100	64	d
0000 0100	4	4	EOT	传输结束	0010 0100	36	24	\$	0100 0101	69	45	E	0110 0101	101	65	e
0000 0101	5	5	ENQ	请求	0010 0101	37	25	%	0100 0110	70	46	F	0110 0110	102	66	f
0000 0110	6	6	ACK	收到通知	0010 0110	38	26	&	0100 0111	71	47	G	0110 0111	103	67	g
0000 0111	7	7	BEL	响铃	0010 0111	39	27	'	0100 1000	72	48	H	0110 1000	104	68	h
0000 1000	8	8	BS	退格	0010 1000	40	28	(	0100 1001	73	49	I	0110 1001	105	69	i
0000 1001	9	9	HT	水平制表符	0010 1001	41	29	)	0100 1010	74	4A	J	0110 1010	106	6A	j
0000 1010	10	0A	LF	换行键	0010 1010	42	2A	*	0100 1011	75	4B	K	0110 1011	107	6B	k
0000 1011	11	0B	VT	垂直制表符	0010 1011	43	2B	+	0100 1100	76	4C	L	0110 1100	108	6C	l
0000 1100	12	0C	FF	换页键	0010 1100	44	2C	,	0100 1101	77	4D	M	0110 1101	109	6D	m
0000 1101	13	0D	CR	回车键	0010 1101	45	2D	-	0100 1110	78	4E	N	0110 1110	110	6E	n
0000 1110	14	0E	SO	不用切换	0010 1110	46	2E	.	0100 1111	79	4F	O	0110 1111	111	6F	o
0000 1111	15	0F	SI	启用切换	0010 1111	47	2F	/	0101 0000	80	50	P	0111 0000	112	70	p
0001 0000	16	10	DLE	数据链路转义	0011 0000	48	30	0	0101 0001	81	51	Q	0111 0001	113	71	q
0001 0001	17	11	DC1	设备控制1	0011 0001	49	31	1	0101 0010	82	52	R	0111 0010	114	72	r
0001 0010	18	12	DC2	设备控制2	0011 0010	50	32	2	0101 0011	83	53	S	0111 0011	115	73	s
0001 0011	19	13	DC3	设备控制3	0011 0011	51	33	3	0101 0100	84	54	T	0111 0100	116	74	t
0001 0100	20	14	DC4	设备控制4	0011 0100	52	34	4	0101 0101	85	55	U	0111 0101	117	75	u
0001 0101	21	15	NAK	拒绝接收	0011 0101	53	35	5	0101 0110	86	56	V	0111 0110	118	76	v
0001 0110	22	16	SYN	同步空闲	0011 0110	54	36	6	0101 0111	87	57	W	0111 0111	119	77	w
0001 0111	23	17	ETB	传输块结束	0011 0111	55	37	7	0101 1000	88	58	X	0111 1000	120	78	x
0001 1000	24	18	CAN	取消	0011 1000	56	38	8	0101 1001	89	59	Y	0111 1001	121	79	y
0001 1001	25	19	EM	介质中断	0011 1001	57	39	9	0101 1010	90	5A	Z	0111 1010	122	7A	z
0001 1010	26	1A	SUB	替补	0011 1010	58	3A	:	0101 1011	91	5B	[	0111 1011	123	7B	{
0001 1011	27	1B	ESC	溢出	0011 1011	59	3B	;	0101 1100	92	5C	\	0111 1100	124	7C	
0001 1100	28	1C	FS	文件分割符	0011 1100	60	3C	<	0101 1101	93	5D	]	0111 1101	125	7D	}
0001 1101	29	1D	GS	分组符	0011 1101	61	3D	=	0101 1110	94	5E	^	0111 1110	126	7E	~
0001 1110	30	1E	RS	记录分离符	0011 1110	62	3E	>	0101 1111	95	5F	_				
0001 1111	31	1F	US	单元分隔符	0011 1111	63	3F	?	0110 0000	96	60	`				
0111 1111	127	7F	DEL	删除	0100 0000	64	40	@								